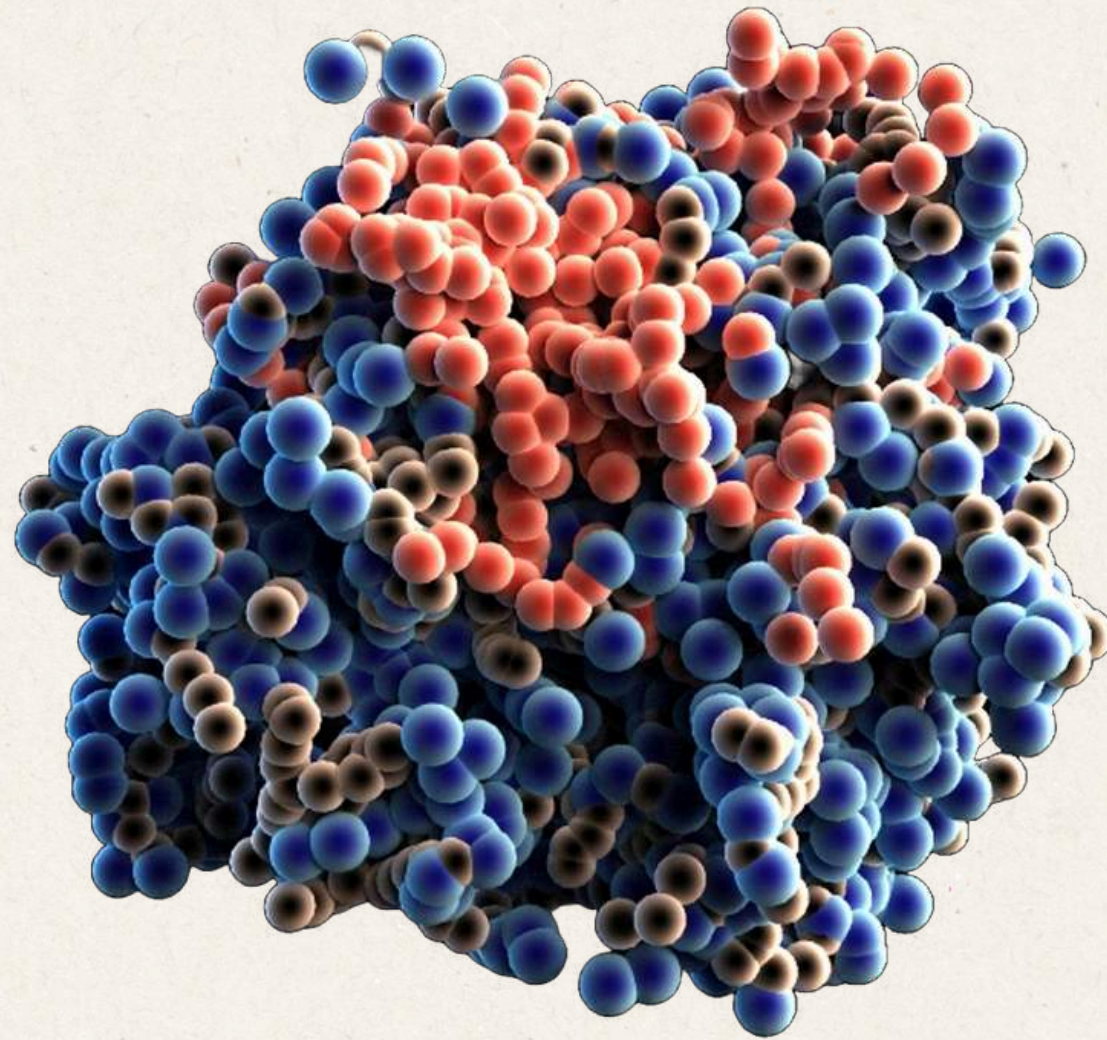


Date: 18th July, 2025



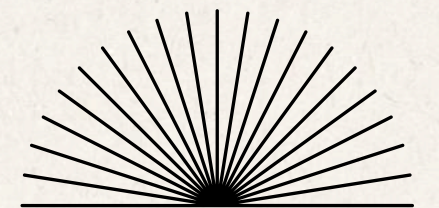
DECODING SNAKE VENOM PROTEINS

Semester Project – Data Science Workflow / Applications

https://github.com/lukedrichard/Snake_Venom_Project

PRESENTED BY:

Saad Tozibar Rahman
Zafrin Sultana
Sakib Hasan
Luke Richard



Agenda

Motivation, Problem Definition

Data Collection, Data Preparation

Model Implementation 1

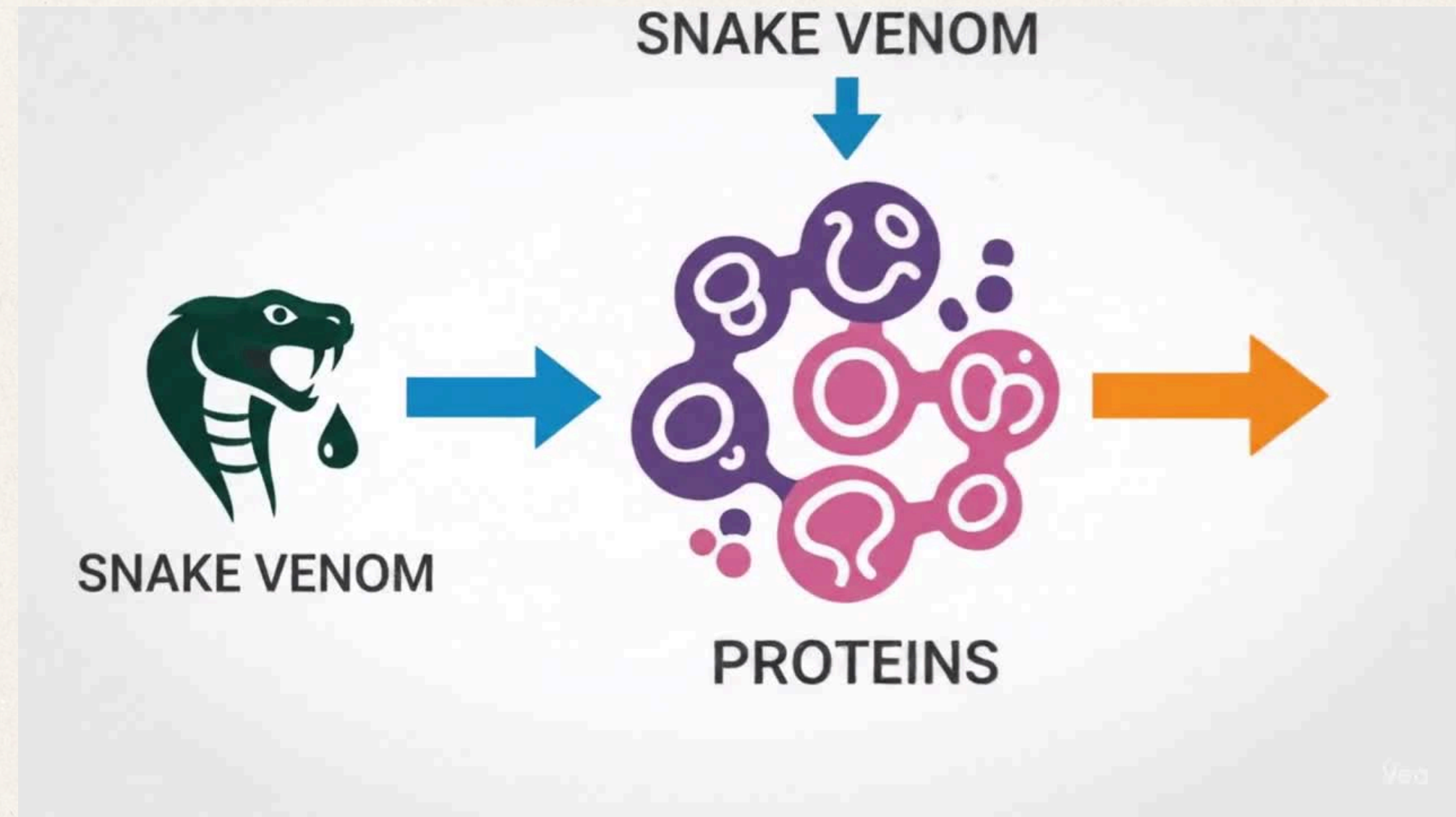
Model Implementation 2

Result

Future Works

Motivation (Problem Setting)

- Snake venom contains biologically active proteins with medical potential.
- Applications in anti-coagulants, painkillers, and antivenoms.
- Manual classification is slow and requires domain expertise.
- Machine learning offers scalable and automated classification.



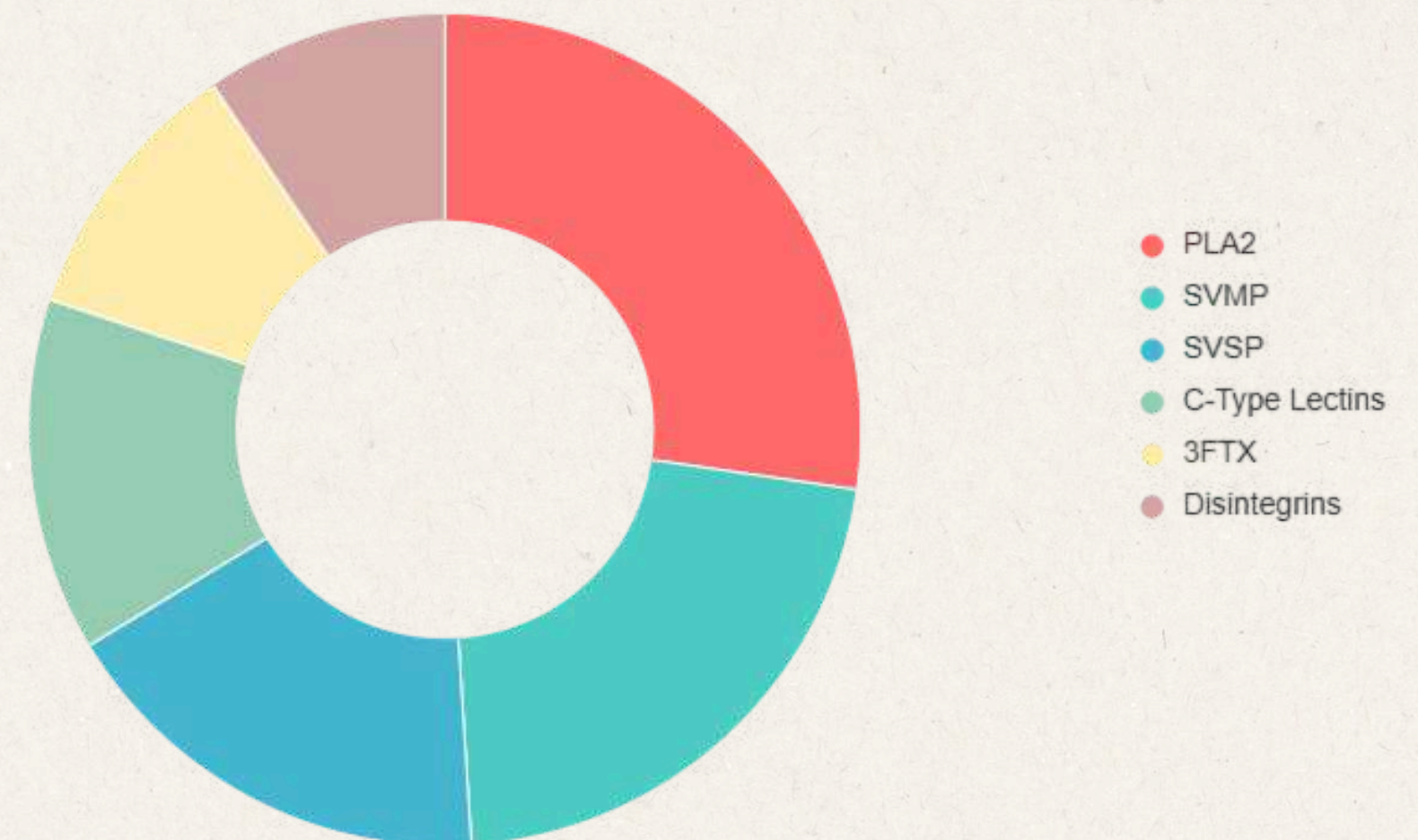
Problem Definition: What Are We Solving?

- **Goal:** Classify venom protein sequences into 6 protein families.
- **Source Data:** Raw protein sequences in FASTA format.
- **Output:** Predicted protein families.

Protein Classes:

1. Serine Proteases (SVSP) - 1102
2. Metalloproteinases (SVMP) - 1378
3. Phospholipase A2 (PLA2) - 1735
4. C-Type Lectins - 873
5. Three-Finger Toxins (3FTX) - 668
6. Disintegrins - 599

Distribution of Venom Protein Sequences



* Kept classes with sequence count around 600+

Data Collection

Source: [UniProt](#) – leading protein database, and a trusted source for curated protein sequences.

Query Design

- **Taxonomy ID:** 8570 (Serpentes)
- **Keywords:** venom OR toxin
- **Protein Name Filters:** e.g., phospholipase, PLA2, crisp, cysteine

Fasta Extraction

- Custom Python script to extract .fasta files.
- Search queries manually created for optimal results.

Search Format: **Taxonomy:** 8570

AND **ALL:** *venom* OR *toxin*

AND (**protein_name:***cysteine* OR **protein_name:***crisp*)

Example: (taxonomy_id:8570) AND (venom OR toxin) AND (protein_name:phospholipase OR protein_name:pla2)

Data Preparation

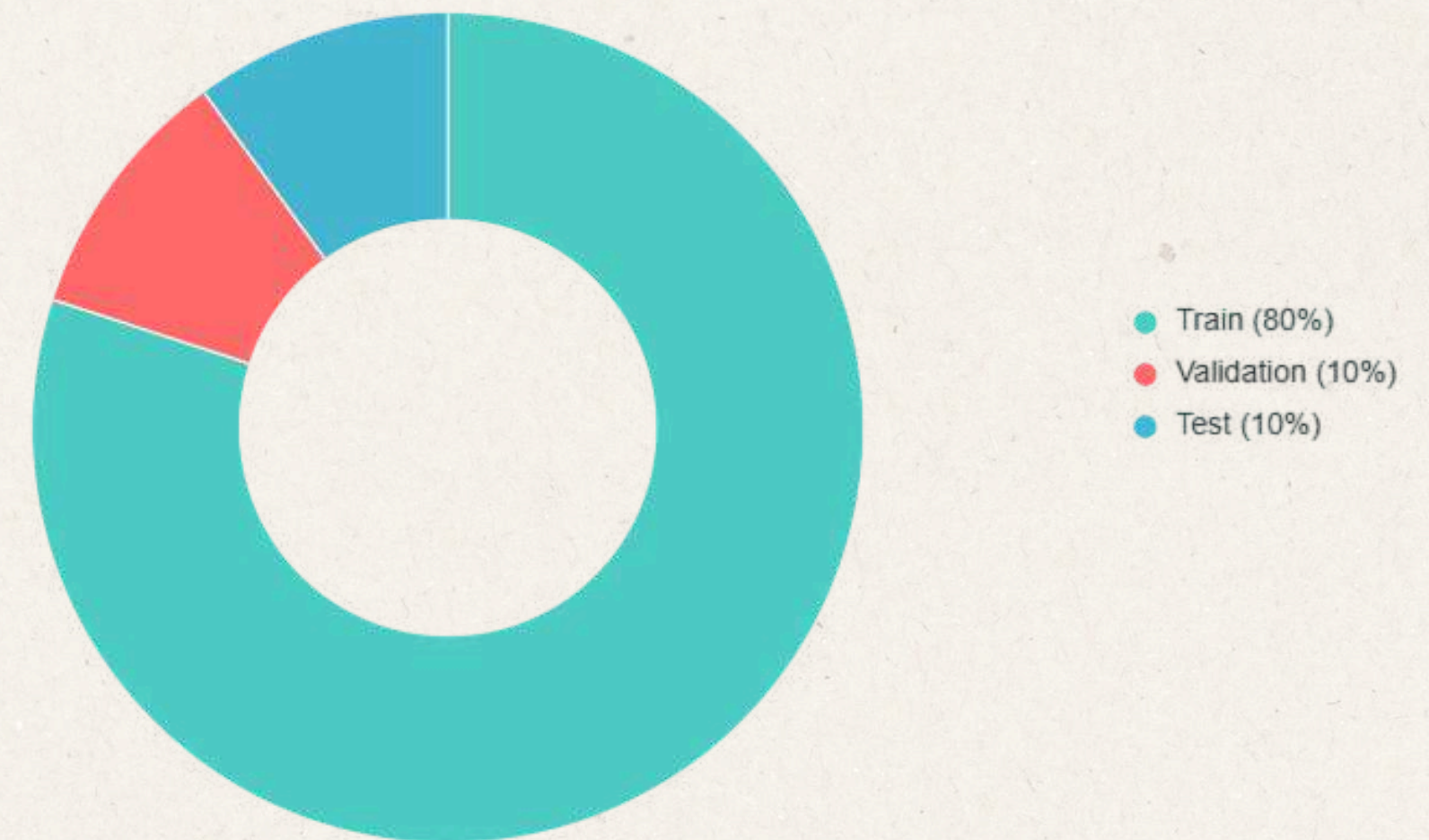
- **Steps:**

- Parse .fasta files into sequences and metadata.
- Assign protein class labels.
- Store clean data in CSV format.

- **Split Ratio:**

- 80% Train, 10% Validation, 10% Test

Dataset Split for Protein Classification



Deduplication with CD-HIT

- Removes similar protein sequences
- Creates clusters based on similarity of k-mers
- Keeps longest sequence as cluster representative
- Similarity Score: 80%
- Ensures test set is truly unseen

Protein	Raw Count	Deduplicated Count
Serine Protease(SVSP)	1102	291
Snake Venom Metalloproteinases(SVMP)	1378	372
Phospholipase A2 (PLA2)	1735	465
C-Type Lectins	873	356
Three Finger Toxin (3FTX)	668	232
Disintegrins	599	218

Introduction to K-mers in Protein Sequences

What is a K-mer?

- A k-mer is a short, continuous subsequence of length k from a biological sequence.
- In proteins, sequences are made of 20 amino acids.

Why K-mers for Protein Feature Extraction?

- Machine learning requires numeric input, not raw strings,
- k-mer frequencies help convert sequences to feature vectors.
- K-mers capture local patterns that reflect biological function or structure.
- Similar to n-grams in NLP, they help the model learn from "motifs" in sequences.

Amino-Acid	3-Letter Name	Symbol
Alanine	Ala	A
Arginine	Arg	R
Asparagine	Asn	N
Aspartic Acid	Asp	D
Asparagine or Aspartic Acid	Asx	B
Cysteine	Cys	C
Glutamine	Gln	Q
Glutamic Acid	Glu	E
Glutamine or Glutamic Acid	Glx	Z
Glycine	Gly	G
Histidine	His	H
Isoleucine	Ile	I
Leucine	Leu	L
Lysine	Lys	K
Methiodine	Met	M
Phenylalanine	Phe	F
Proline	Pro	P
Serine	Ser	S
Threonine	Thr	T
Tryptophan	Trp	W
Tyrosine	Tyr	Y
Valine	Val	V

20 Standard Amino Acids

K-mer Feature Generation Pipeline

- Load raw protein sequences from CSV.
- Define k-mer types: single (1-mer), di (2-mer), tri (3-mer).
- Clean sequences to remove unknown amino acids.
- Generate frequency vectors for each k-mer type.
- Save processed features to CSV.
- Merge features with metadata (train/test/val fold).
- Save split datasets for ML training.

Example Protein sequence: MIEVL

→ 1-mers: M, I, E, V, L

→ 2-mers: MI, IE, EV, VL

→ 3-mers: MIE, IEV, EVL

1-mer Frequency Table :

A	C	D	E	F	G	H	I	K	L	M	N	P	Q	R	S	T	V	W	Y
0	0	0	0.2	0	0	0	0.2	0	0.2	0.2	0	0	0	0	0	0.2	0	0	0

2-mer Frequency Table:

MI	IE	EV	VL	...
0.25	0.25	0.25	0.25	...

Model Selection with Randomized Hyperparameter Search

Model Selection Process

- Built a modular pipeline using a custom class: ***MultiModelRandomSearch***
- Purpose:
 - Automate training, hyperparameter tuning, and evaluation
 - Support multiple models in one framework
- Compared two classifiers:
 - Random Forest
 - AdaBoost

```
self.param_distributions = {  
    "RandomForest": {  
        "n_estimators": randint(100, 300),  
        "max_depth": [None, 10, 20, 30],  
        "min_samples_split": randint(2, 10),  
        "min_samples_leaf": randint(1, 5),  
        "bootstrap": [True, False]  
    },  
    "AdaBoost": {  
        "n_estimators": randint(50, 150),  
        "learning_rate": uniform(0.1, 1.0)  
    }  
}
```

Hyperparameter Tuning Details

- Used RandomizedSearchCV for efficiency
- 20 Random combination, 5 folds cross validation ,Scoring based on accuracy

```
RandomForest - Accuracy: 0.7371  
Best Params: {'bootstrap': False, 'max_depth': None, 'min_samples_leaf': 1, 'min_samples_split': 8, 'n_estimators': 221}
```

```
AdaBoost - Accuracy: 0.7165  
Best Params: {'learning_rate': 1.0737555188414591, 'n_estimators': 64}
```

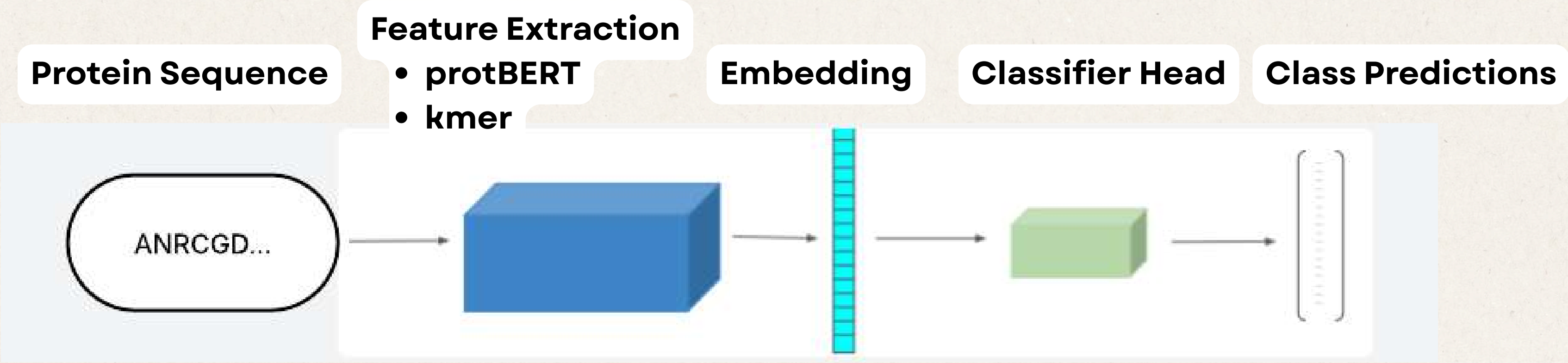

Final Model Evaluation

FinalModelEvaluator: Custom class for evaluating classification models using K-Fold cross-validation.

- Initialize the FinalModelEvaluator with the best model.
- Use 5-fold cross-validation to test model performance on different data splits
- Evaluate metrics: accuracy, precision, recall, f1-score, and roc_auc (if supported)
- For each fold, train and predict, then compute and plot the confusion matrix
- Summarize results by calculating mean and standard deviation of all metrics across folds

```
Evaluating on Test Set:  
Accuracy: 0.7617  
Precision: 0.7564  
Recall: 0.7617  
F1 Score: 0.7568  
ROC-AUC Score (OVR): 0.8961
```


Deep Learning: Setup

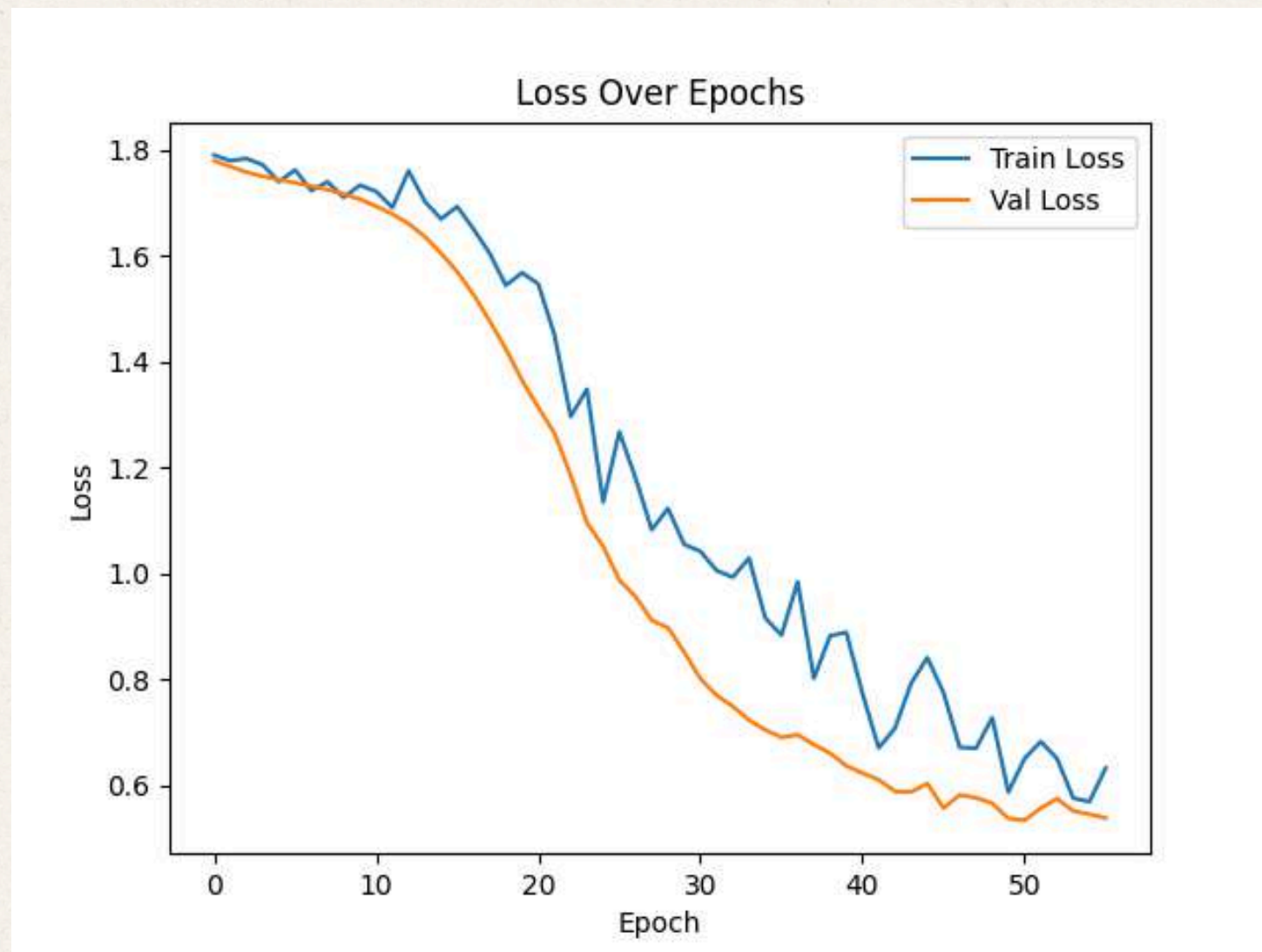


• • • • •
• • • • •
• • • • •
• • • • •
• • • • •

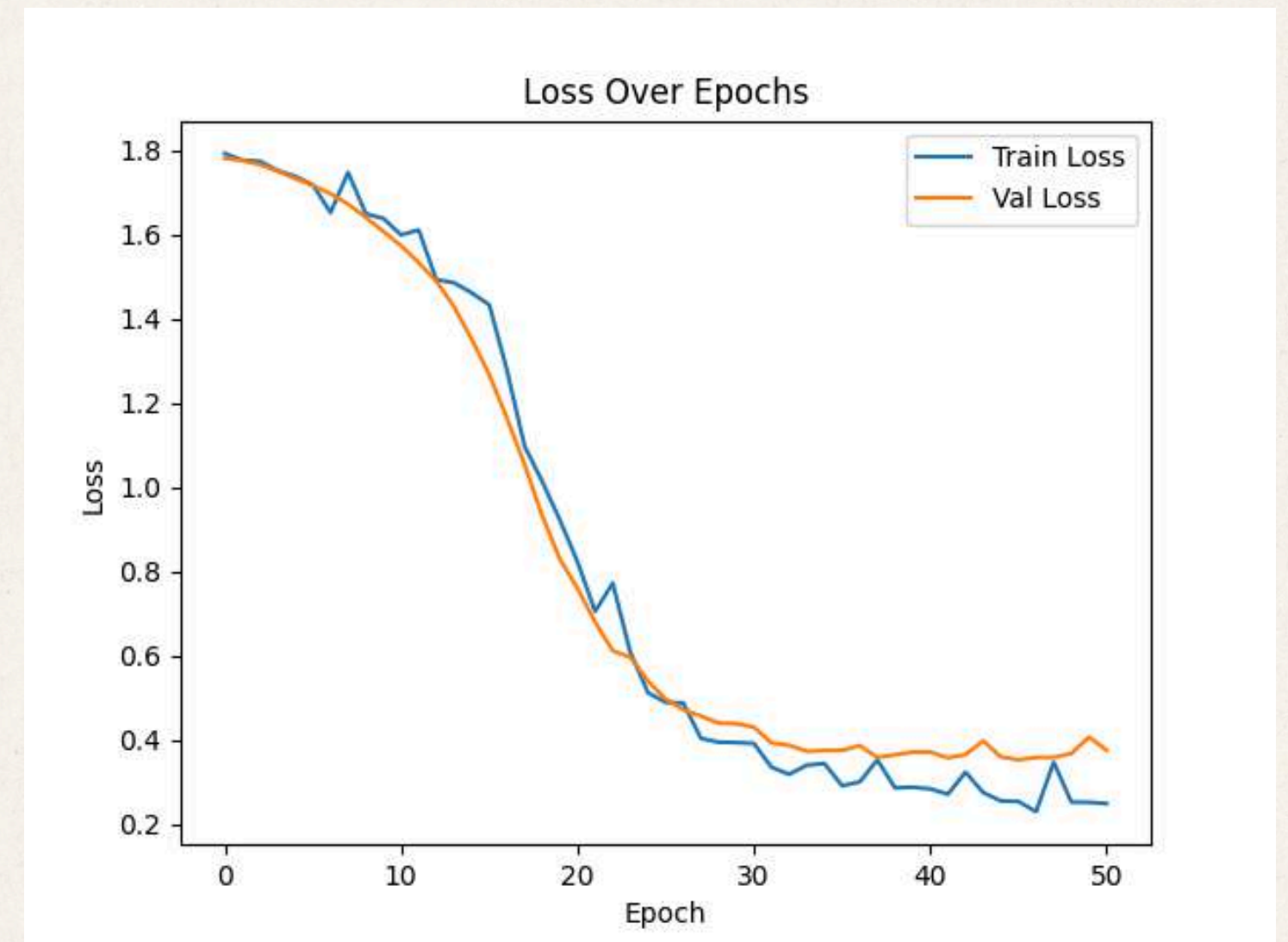
- protBERT trained for masked protein sequence modeling
- Classifier: 4 Fully Connected Linear Layers(MLP)
- Cross-Entropy Loss
- Adam Optimizer
- Learning Rate = 1e-3
- Dropout = 0.7
- Early Stopping with Patience = 5

Training

Post CD-HIT Deduplication

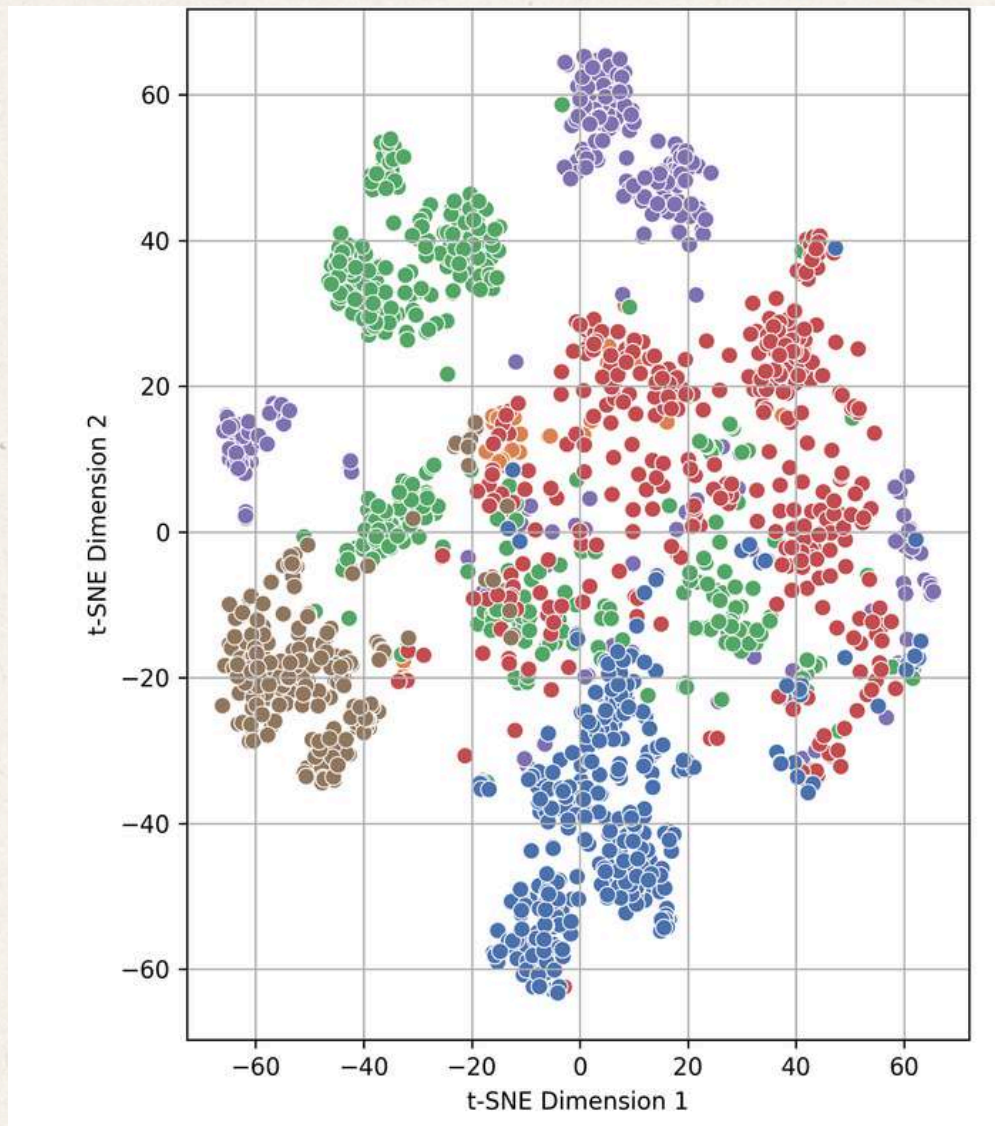


Deduplicated kmer

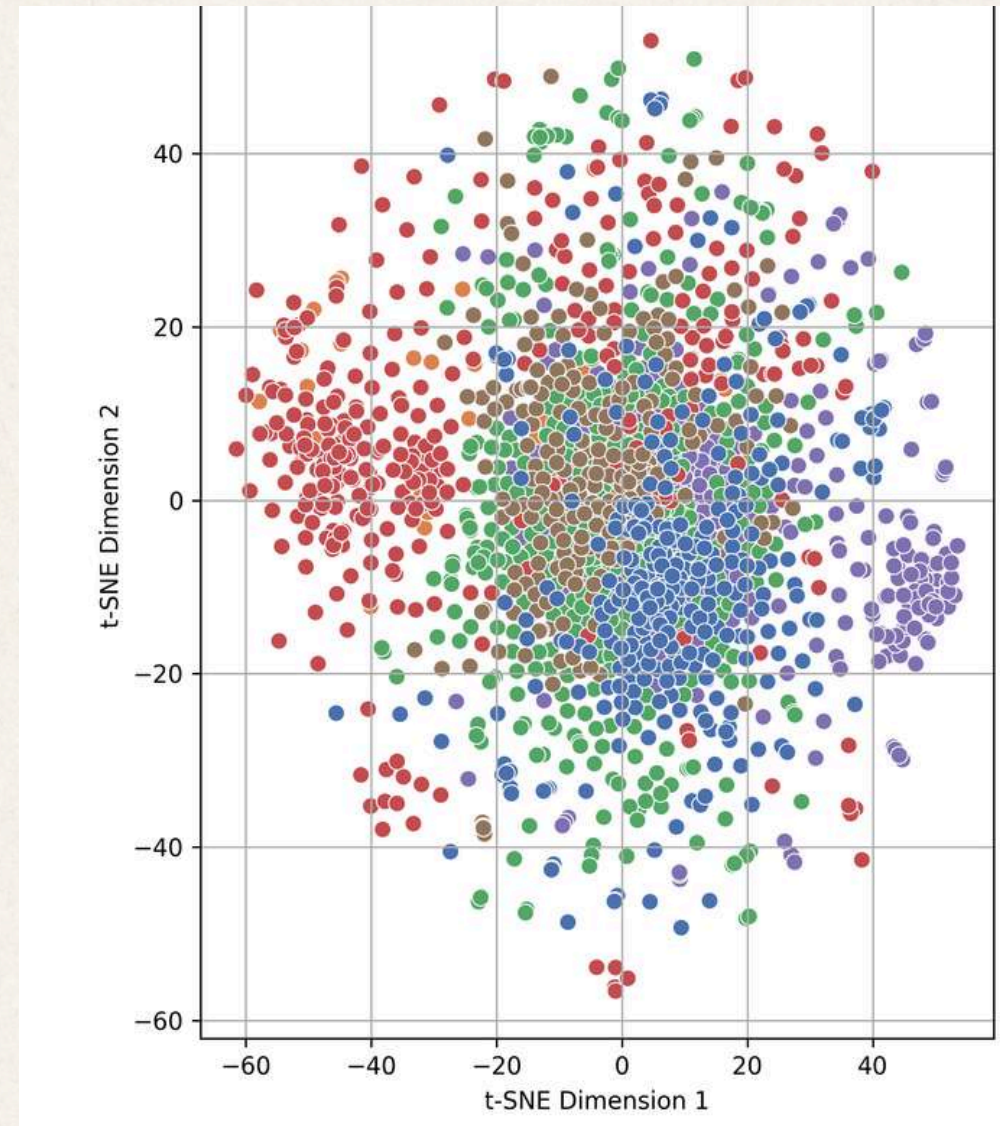


Embedding Visualization with t-SNE

protBERT(deduplicated)



Kmer Features(deduplicated)



Legend

Protein Class	
●	Snake_Venom_Serine_Proteases(SVSP)
●	Snake_Venom_Metalloproteinases(SVMP)
●	Phospholipase_A2(PLA2)
●	C-Type_Lectins_or_Lectin-Like_Proteins
●	Three-Finger_Toxins(3FTX)
●	Disintegrins

- protBERT Embeddings are well clustered
- SVMs and Disintegrins have a lot of overlap
- Kmer features are sparse

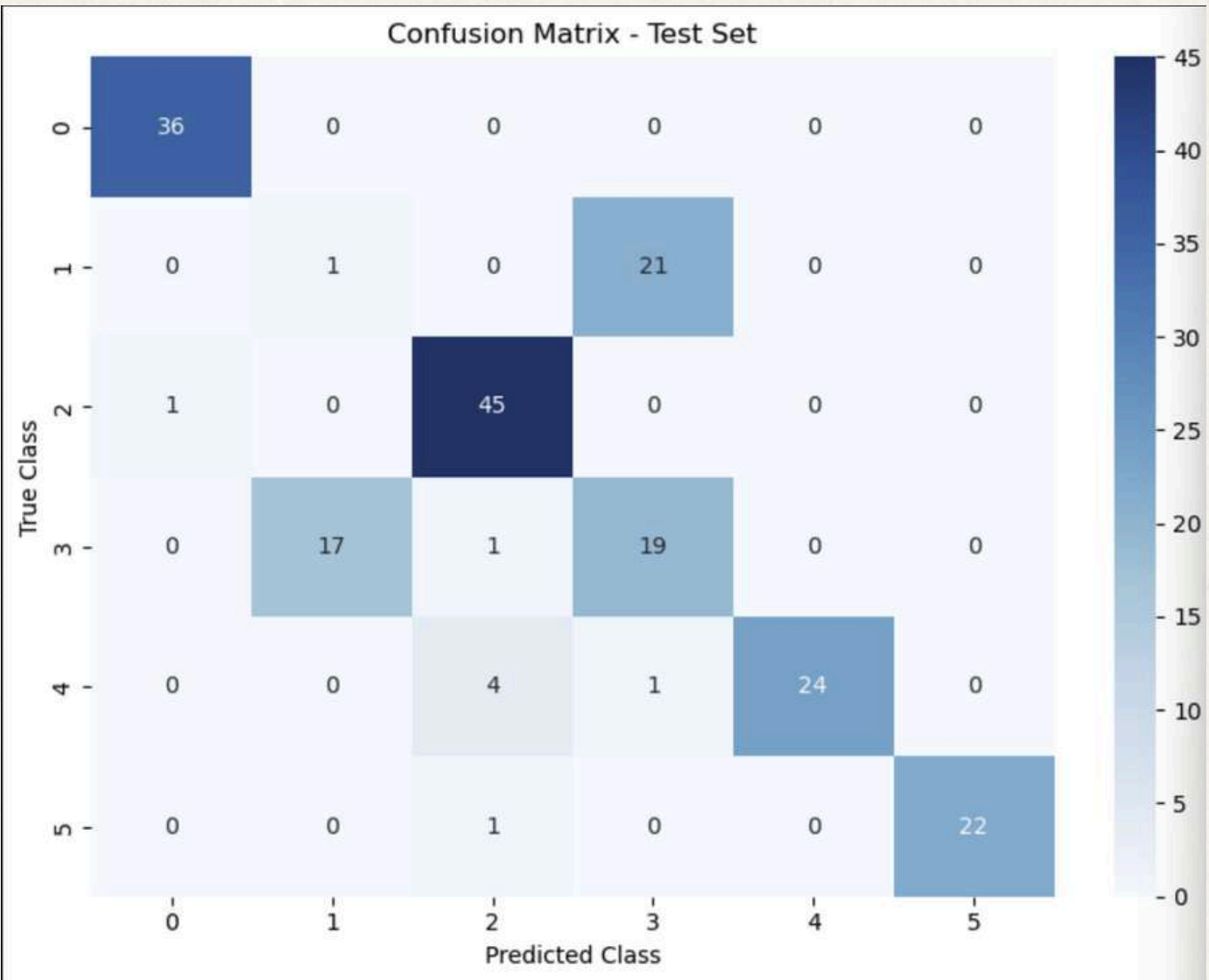
Evaluation - Scores

Model	Accuracy	Precision	Recall	F1 Score	ROC AUC
Random Forest (5-fold CV)	0.7617	0.7564	0.7617	0.7568	0.8961
MLP (deduplicated ProtBERT)	0.8031	0.7381	0.8031	0.7628	0.9682
MLP (deduplicated k-mer)	0.8601	0.8642	0.8601	0.8309	0.9715

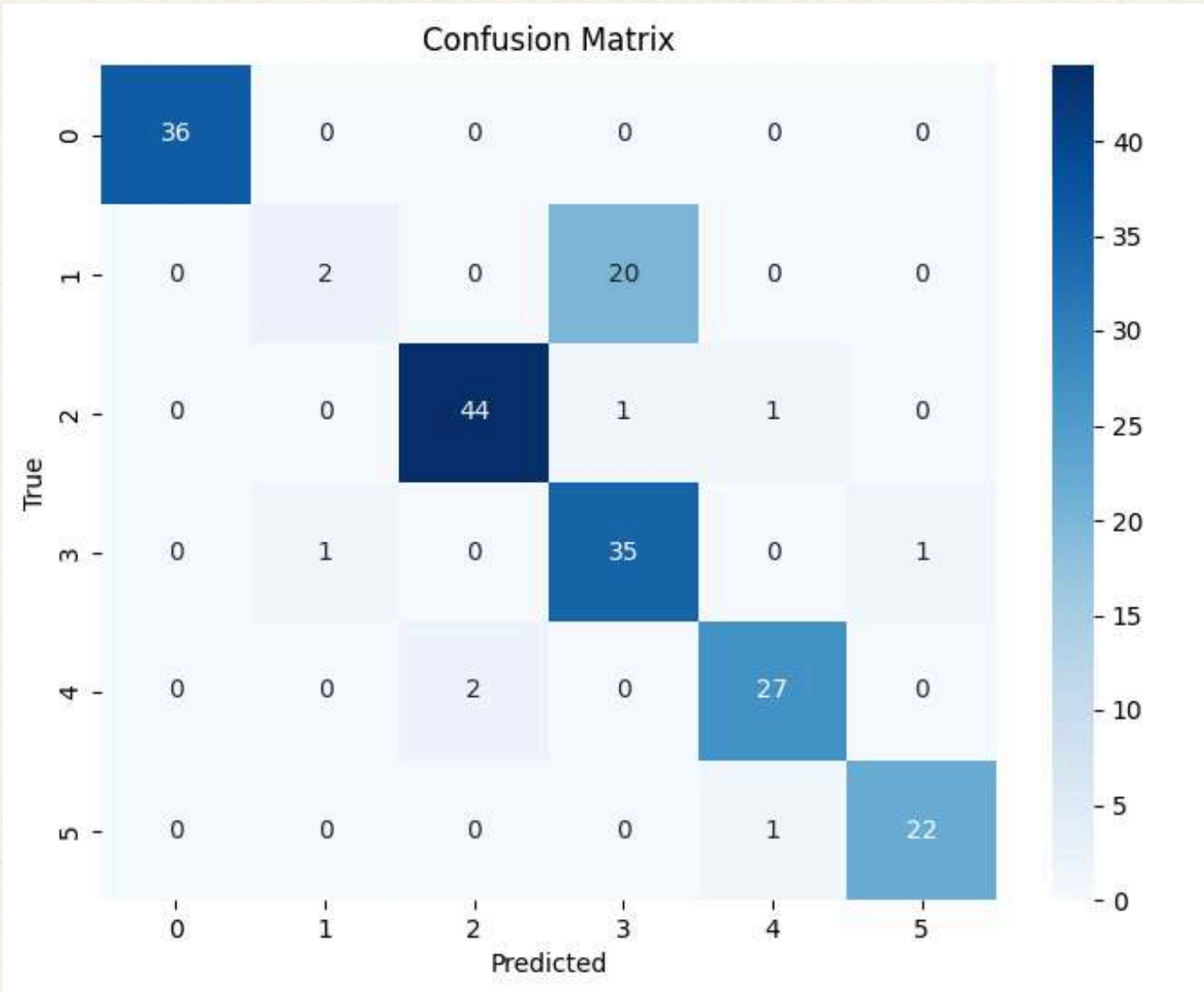
• • • • • • • • • •
• • • • • • • • • •
• • • • • • • • • •
• • • • • • • • • •
• • • • • • • • • •
• • • • • • • • • •

- Other than accurcay, all metrics were **weighted** to handle class imbalance, ensuring fair evaluation.
- Models **generalized** well, with strong performance across most classes.
- K-mer embedding **outperformed** ProtBert, as it better captured local patterns, while ProtBert’s [CLS] token was too general.

Evaluation - Confusion Matrices



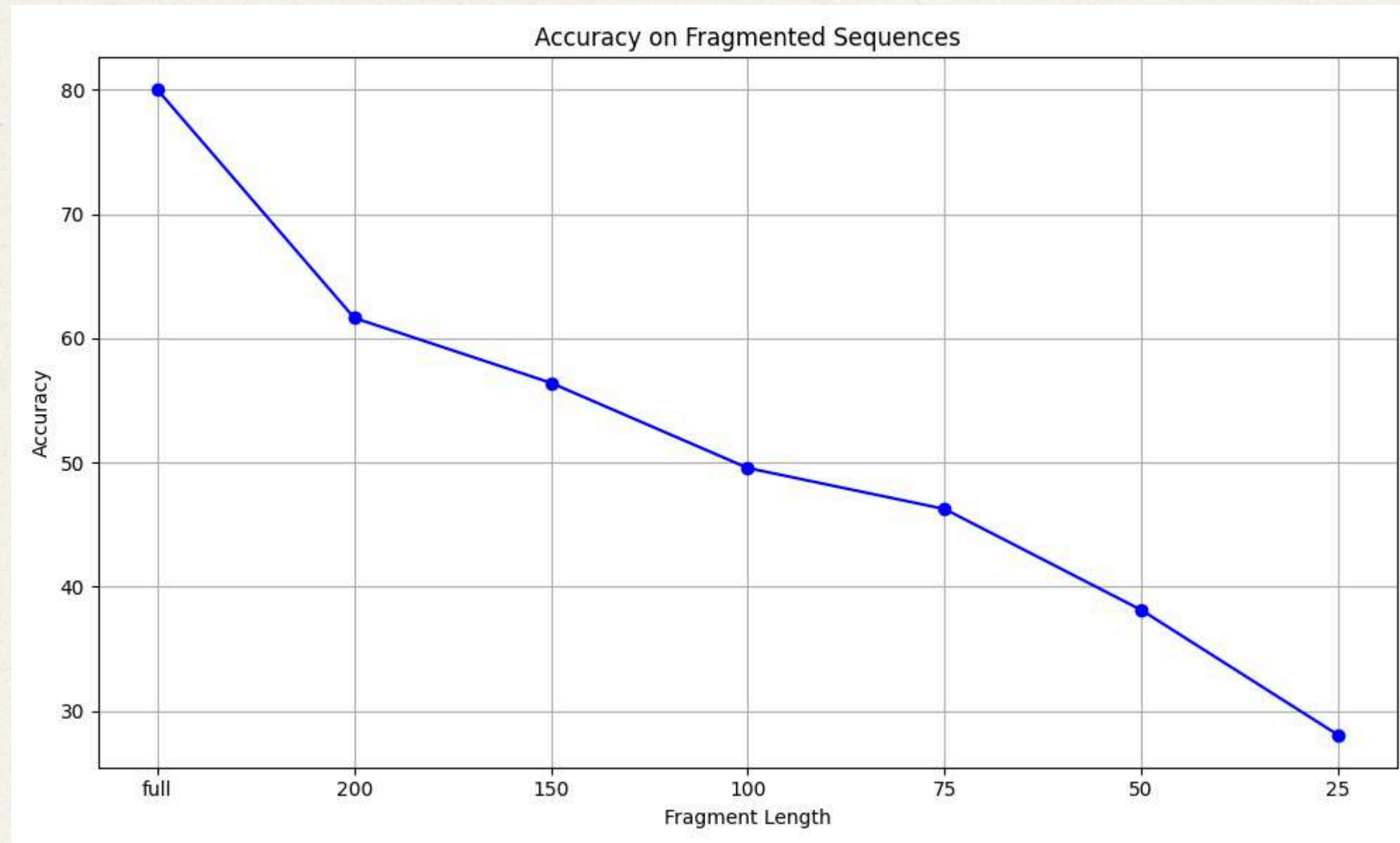
Random Forest (5-Fold)



MLP Classifier

class 1: Disintegrins(Zinc Metalloproteinases)
class 3: Snake Venom Metalloproteinases

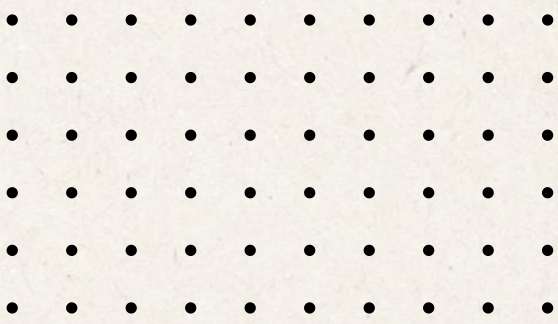
Evaluation - Robustness



- Sliced test set sequences into fragments of varying lengths
- Model quickly loses accuracy with shorter fragments
- Could add data augmentation in testing

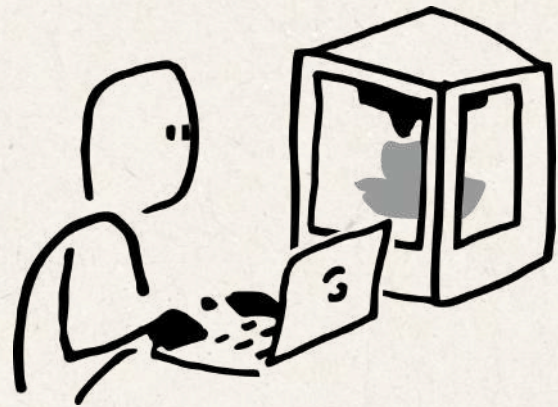
Conclusions

- MLP (deduplicated k-mer) **outperforms** all models.
- Random Forest performs well for a classical model but **lags in F1 and accuracy**.
- MLP with ProtBERT embedding showed **improved results over Random Forest** but underperformed compared to k-mer MLP.
- Random Forest is **less competitive** compared to deep learning approaches on this task.
- **ROC AUC remains high** across all models, indicating robust classification ability.
- The main **challenge remain** across models is the **confusion** between similar classes (**Class 1 vs 3**)



Future Work

- Apply **additional feature extraction** methods, such as g-gap dipeptide decomposition.
- **Improve ProtBert embedding** usage by using the full embeddings instead of only the [CLS] vector.
- **Train 1D CNNs** on the full ProtBert embedding arrays to capture positional and contextual patterns.
- Use **advanced models like transformers or attention-based architectures** for deeper sequence understanding.
- **Improve class separation**, (Class 1 Vs 3) using advanced modeling techniques or biologically informed pre-processing.



Thank you